

ABC458 D - Chalkboard Median 题解

题意概括

黑板上初始有一个整数 x 。

第 i 次询问加入两个整数 A_i 、 B_i 后，黑板上总共有 $2i + 1$ 个数。要求输出这些数的中位数。

解题思路

这是一个典型的“动态维护中位数”问题。

用两个堆维护当前所有数：

- `left`：大根堆，保存较小的一半
- `right`：小根堆，保存较大的一半

并始终保持下面两个性质：

1. `left` 中的所有数都不大于 `right` 中的所有数；
2. `left` 的元素个数恰好比 `right` 多 1。

这样一来：

- 当前总数一定是奇数；
- 中位数就是 `left` 的堆顶。

每次加入一个新数时：

1. 如果它不大于当前中位数，就先放进 `left`；
2. 否则放进 `right`；
3. 再通过移动堆顶，把两个堆重新调整到上面的平衡状态。

每次询问会加入两个数，连续执行两次插入和平衡即可。

正确性说明

我们始终维护：

1. `left` 中所有数都不大于 `right` 中所有数；
2. $|\text{left}| = |\text{right}| + 1$ 。

由于总元素个数始终为奇数，所以：

- left 恰好包含较小的那一半再多一个;
- right 恰好包含较大的那一半。

因此:

- left 中最大的数, 也就是 left.top(),
- 恰好是排序后的正中间那个数,

也就是中位数。

每次插入后, 我们只可能暂时破坏 “大小关系” 或 “数量关系” :

- 若大小关系被破坏, 把不该在左边的最大值移到右边, 或把不该在右边的最小值移到左边;
- 若数量关系被破坏, 再把堆顶移动一次恢复平衡。

调整完成后, 不变式重新成立, 所以每次输出的 left.top() 都是正确的中位数。算法正确。

复杂度

每次询问插入两个数, 每次插入与平衡的代价都是 $O(\log Q)$ 。

- 时间复杂度: $O(Q \log Q)$
- 空间复杂度: $O(Q)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;

int main(){

    int x, q;
    cin >> x >> q;

    priority_queue<int> left;
    priority_queue<int, vector<int>, greater<int>> right;

    auto rebalance = [&]() {
        while(!right.empty() && left.top() > right.top()){
            int a = left.top();
            int b = right.top();
            left.pop();
            right.pop();
        }
    };

    while(q--){
        if(x > 0){
            left.push(x);
            rebalance();
        } else {
            right.push(-x);
            rebalance();
        }
        x = 0;
    }

    cout << left.top() << endl;
}
```

```

        left.push(b);
        right.push(a);
    }

    while((int)left.size() < (int)right.size() + 1){
        left.push(right.top());
        right.pop();
    }

    while((int)left.size() > (int)right.size() + 1){
        right.push(left.top());
        left.pop();
    }
};

auto add_value = [&](int v){
    if(left.empty() || v <= left.top()){
        left.push(v);
    } else {
        right.push(v);
    }
    rebalance();
};

left.push(x);

while(q--){
    int a, b;
    cin >> a >> b;

    add_value(a);
    add_value(b);

    cout << left.top() << '\n';
}

return 0;
}

```